A Supplementary Materials for Methodology

A.1 PPO Surrogate Objectives

Discrete Action Policy:

The surrogate objective $\mathcal{L}_d^{\text{PPO}}$ for the discrete policy $\pi_{\varsigma_d}(\cdot)$ is defined as:

$$\begin{aligned}\mathbb{E}_{(\mathbf{s},a)\sim\pi_{\text{old}}}\left[\min\left(\kappa A(\mathbf{s},a), \text{clip}\left(\kappa, 1-\epsilon, 1+\epsilon\right) A(\mathbf{s},a)\right)\right] \\ +\beta_d \mathbb{E}_\pi\left[H\left(\pi_{\varsigma_d}(a|s)\right)\right],\end{aligned} \tag{9}$$

where π_{old} represents the frozen parameters of the policy from the previous iteration. $\kappa = \frac{\pi_{\varsigma_d}(a|\mathbf{s})}{\pi_{\text{old}}(a|\mathbf{s})}$. $A(\mathbf{s},a)$ is the advantage function for discrete actions, computed via Generalized Advantage Estimation (GAE). clip$(\cdot)$ restricts the policy ratio to $[1-\epsilon, 1+\epsilon]$ to prevent destabilizing policy updates. β_d is the entropy regularization coefficient, and $H(\cdot)$ measures the policy entropy to encourage exploration.

Continuous Action Policy:

The surrogate objective $\mathcal{L}_c^{\text{PPO}}$ for the continuous policy $\pi_{\varsigma_c}(\cdot)$ is defined as:

$$\begin{aligned}\mathbb{E}_{(\mathbf{s},a,z)\sim\pi_{\text{old}}}\left[\min\left(\Gamma A(\mathbf{s},a,z), \text{clip}\left(\Gamma, 1-\epsilon, 1+\epsilon\right) A(\mathbf{s},a,z)\right)\right] \\ +\beta_c \mathbb{E}_\pi\left[H\left(\pi_{\varsigma_c}(z|\mathbf{s},a)\right)\right],\end{aligned} \tag{10}$$

where $A(\mathbf{s},a,z)$ incorporates the joint effect of discrete and continuous actions on the advantage estimation. $\Gamma = \frac{\pi_{\varsigma_c}(z|\mathbf{s},a)}{\pi_{\text{old}}(z|\mathbf{s},a)}$. β_c is the entropy regularization coefficient.

Advantage Function Computation:

The advantage functions $A(\mathbf{s},a)$ and $A(\mathbf{s},a,z)$ are derived from the critic policy network $V_\varphi(\cdot)$ using GAE:

$$A(\mathbf{s}^t,a^t,z^t) = \sum_{l=0}^{T-t}(\gamma\lambda)^l\delta^{t+l}, \quad \delta^t = r^t + \gamma V_\varphi(\mathbf{s}^{t+1}) - V_\varphi(\mathbf{s}^t), \tag{11}$$

with $\gamma \in [0,1)$ as the discount factor and $\lambda \in [0,1]$ as the GAE smoothing coefficient.

B Experiment Settings

B.1 Evaluation Tasks

For the graph-level task in the few-shot scenario, we follow previous works [3, 12] to adopt the 50-shot setting to ensure experimental fairness. For the node-level task, inspired by previous works [24, 26], we generate induced graphs for nodes, enabling universal prompt tuning to be extended to node-level tasks by operating on n-hop subgraphs. For the node-level task in the few-shot scenario, we follow previous work [42] to adopt the 10-shot setting to ensure experimental fairness. This ensures an appropriate number of nodes in the induced subgraphs.

B.2 Model Architectures

We follow previous works [3, 12, 42] to adopt 5-layer GIN and 2-layer GIN for graph- and node-level tasks, respectively, to ensure experimental fairness. Hidden dimensions for GIN are set as 300 and 128 for graph- and node-level tasks, respectively. We adopt weight decay $5e^{-4}$ for the projection head in the graph-level task and utilize Singular Value Decomposition (SVD) to reduce the initial features of each dataset to 100 dimensions for the node-level task. In addition, there are some consistent settings for both graph- and node-level tasks. Since our RL approach differs from RELIEF, which requires generating universal graph prompts independently, we only need to edit basic universal graph prompts. As a result, our RL approach is not highly sensitive to hyper-parameters. For all scenarios, we keep the relevant hyper-parameters fixed. Specifically, we set 0.5 for dropout rate, $5e^{-4}$ for learning rate of actors and critic, 0.99 for discount factor γ, 0.95 for the GAE smoothing coefficient λ, 0.2 for PPO clip range, 0.01 for entropy regularization coefficients β_d and β_c. Parameter settings are summarized in Table 6.

Table 6: Parameter settings for graph- and node-level tasks.

Parameter	Graph-level	Node-level
GIN Layer Number	5	2
GIN Hidden Dimension	300	128
Projection Head Weight Decay	$5e^{-4}$	-
SVD Dimensions	-	100
Dropout Rate	0.5	
Learning Rate for Both Actors and Critic	$5e^{-4}$	
Discount Factor γ	0.99	
GAE Smoothing Coefficient λ	0.95	
PPO Clip Range	0.2	
Entropy Regularization Coefficients β_d and β_c	0.01	

B.3 Datasets

For the graph-level task, we follow previous works [3, 12, 42] to utilize Chemical datasets[5] [8] to pre-train GNN models. These datasets consist of both unlabeled and labeled molecules, enabling comprehensive model training. Specifically, 2 million unlabeled molecules from the ZINC15 database are employed for node-level self-supervised pre-training. For graph-level multi-task supervised pre-training, a preprocessed subset of the ChEMBL database is used, which comprises 456k labeled molecules. This subset provides a diverse chemical space that enhances the model's generalization across various biochemical tasks. For downstream graph classification tasks, we utilize 8 binary classification datasets focused on molecular property prediction related to biophysics and physiology [32]. We split each dataset into training, validation, and testing sets with ratios of 80%, 10%, and 10%, respectively. Molecules are divided based on their scaffolds, i.e., molecular graph substructures [21], and the clusters are reorganized by prioritizing the most frequently occurring scaffolds in the training set. This approach ensures that the validation and testing sets contain structurally distinct molecules [8], enabling an assessment of the model's out-of-distribution generalization capabilities. The statistics of these downstream datasets are summarized in Table 7.

For the graph-level task, we follow previous works [24, 42] to utilize Cora, Citeseer, Pubmed [10], and Amazon-Co-Buy (Computer and Photo) [23], where the first three datasets represent each node as a publication with a sparse bag-of-words feature vector, and edges signify citation links, the last two datasets are segments of the Amazon co-purchase graph, with nodes representing products, edges indicating frequent co-purchases, node features encoded as bag-of-words from product reviews, and class labels derived from product categories. Taking into account the average degree of different datasets, we set varying values of n for generating n-hop

[5] https://github.com/snap-stanford/pretrain-gnns/tree/master/chem/model_gin/.

Table 7: Statistics of the downstream datasets used in the graph-level task.

Datasets	BBBP	Tox21	ToxCast	SIDER	ClinTox	MUV	HIV	BACE
# Molecules	2,039	7,831	8,575	1,427	1,478	93,087	41,127	1,513
# Tasks	1	12	617	27	2	17	1	1

subgraphs: specifically 4, 3, 2, 2, and 2 for Cora, Citeseer, Pubmed, Computers, and Photo, respectively. The statistics of these datasets are summarized in Table 8.

Table 8: Statistics of the datasets used in the node-level task.

Datasets	Cora	Citeseer	Pubmed	Computers	Photos
# Nodes	2,708	3,327	19,717	13,752	7,650
# Edges	5,429	9,104	88,648	491,722	238,162
# Features	1,433	3,703	500	767	745
# Classes	7	6	3	10	8

B.4 Pre-training Strategies

To ensure fair evaluation, we follow previous works [3, 42] by adopting four pre-training strategies for graph-level tasks and two pre-training strategies for node-level tasks. Specifically, for the graph-level task, we employ five widely used strategies to pre-train the graph models, including Deep Graph Infomax (Infomax) [27], Attribute Masking (AttrMasking) [8], Context Prediction (ContextPred) [8], and Graph Contrastive Learning (GCL) [40]:

- **Infomax** Deep Graph Infomax learns expressive representations for graphs or nodes by maximizing the mutual information between graph-level representations and substructure-level representations across different granularities.
- **AttrMasking** Attribute Masking involves masking node or edge attributes and tasking GNNs with predicting the masked attributes based on the surrounding structural information.
- **ContextPred** Context Prediction leverages subgraphs to predict their surrounding graph structures, aiming to map nodes with similar structural contexts to closely aligned embeddings in the representation space.
- **GCL** Graph Contrastive Learning aims to embed augmented versions of an anchor graph closer to each other (positive samples) while pushing apart the embeddings of different graphs (negative samples). For generating positive and negative samples, we adopt the augmentation strategies [40].

For the node-level task, we utilize two widely used strategies to pre-train the graph models: Masked Edge Prediction (MaskedEdge) [24] and Edge Connectivity Contrastive Learning (ContraEdge) [18]:

- **MaskedEdge** Masked Edge Prediction, proposed by GPPT [24], leverages a binary cross-entropy loss to optimize the predicted interaction probabilities between nodes and their adjacency matrix, thereby capturing rich edge information.
- **ContraEdge** Edge Connectivity Contrastive Learning employed by GraphPrompt [18] determines positive and negative node pairs based on edge connectivity, encouraging the model to bring embeddings of positively connected node pairs closer while pushing apart embeddings of negatively connected pairs.

B.5 Baselines

For the graph-level task, we compare LEAP against Fine-tuning (FT), GPF, GPG-plus [3], $\text{SUPT}_{\text{soft}}$, $\text{SUPT}_{\text{hard}}$ [12], and RELIEF [42]. In the node-level task, we include all the above baselines and additionally compare with GPPT [24] for the MaskedEdge pre-training strategy and GPrompt [18] for the ContraEdge pre-training strategy. Specifically, given a pre-trained graph model $f_\theta(\cdot)$ and a task-specific projection head $g_\phi(\cdot)$, FT tunes the parameters of the pre-trained graph model $f_\theta(\cdot)$ and a task-specific projection head $g_\phi(\cdot)$ simultaneously during the downstream training stage. GPF, GPG-plus, $\text{SUPT}_{\text{soft}}$, $\text{SUPT}_{\text{hard}}$, and RELIEF freeze the parameters of the pre-trained model $f_\theta(\cdot)$ and introduce extra prompt vectors p. These baselines tune the parameters of the prompt vectors p and a task-specific projection head $g_\phi(\cdot)$ simultaneously during the downstream training stage. For the node-level task, we include all baselines used in the graph-level task and additionally incorporate GPPT and GraphPrompt as baselines. These methods are specifically designed for graph models pre-trained at the edge level and are well-suited for transfer to node classification. Notably, we did not include All in One [26], as it relies on a frozen graph model specifically pre-trained using GCL.

B.6 Hyper-parameter Settings

We detail the tuning range of all hyper-parameters in our LEAP to enhance the reproducibility. Following previous works [3, 24, 42], we tune training epochs within $[50, 100]$ and $[50, 100, 300]$ for graph- and node-level tasks, respectively. Moreover, there are some consistent hyper-parameter tuning ranges for both graph- and node-level tasks. Specifically, we conduct a grid search for the graph loader batch size within $[8, 16, 32, 64]$, the basic universal graph prompt vector number k within $[5, 10, 20]$, the prompt edit range ϑ within $[0.1, 0.5, 1]$, reward function balancing hyper-parameter λ_c within $[1e^{-5}, 1e^{-4}, 1e^{-3}]$, policy networks update interval h within $[1, 2, 3, 4]$, projection head learning rate within $[5e^{-4}, 1e^{-3}, 5e^{-3}]$, projection head layer number within $[1, 2, 3]$, PPO mini-batch size within $[32, 64, 128, 256]$, and policy finite horizon (step) T within $[N/8, N/4, N/2, N, 2N]$. Hyper-parameter settings are summarized in Table 9.

C Additional Experimental Results

C.1 Few-shot Scenario

We further conduct experiments in the few-shot scenario to validate the effectiveness of our LEAP. Table 10 and Table 11 show that our LEAP achieves superior performance compared to baselines on graph- and node-level tasks in the few-shot scenario, with improvements in 27/32 and 17/20 tasks (metrics), respectively. Notably, RELIEF performs well in the few-shot scenario, but its performance in the full-shot scenario is unsatisfactory, which is consistent with

Table 9: Hyper-parameter settings.

Parameter	Graph-level	Node-level
Training Epochs	[50, 100]	[50, 100, 300]
Graph Loader Batch Size	[8, 16, 32, 64]	
Basic Universal Graph Prompt Vector Number k	[5, 10, 20]	
Prompt Edit Range ϑ	[0.1, 0.5, 1]	
Reward Balancing Parameter λ_c	$[1e^{-5}, 1e^{-4}, 1e^{-3}]$	
Policy Networks Update Interval h	[1, 2, 3, 4]	
Projection Head Learning Rate	$[5e^{-4}, 1e^{-3}, 5e^{-3}]$	
Projection Head Layer Number	[1, 2, 3]	
PPO Mini-batch Size	[32, 64, 128, 256]	
Policy Finite Horizon (Step) T	$[N/8, N/4, N/2, N, 2N]$	

the conclusion we derived in Theorem 1. Its selective node-based graph prompt tuning facilitates prompt learning but undermines the theoretical foundation of universal graph prompt tuning.

C.2 Ablation Study

To analyze the effectiveness of LEAP, we conduct extensive ablation studies to evaluate the necessity and contribution of each individual component within the model. Specifically, we compare our model with the following variants: 1) LEAP_{GPF}, which replaces the basic universal graph prompt with GPF. 2) $\text{LEAP}_{\text{GPF-plus}}$, which replaces the basic universal graph prompt with full GPF-plus. 3) LEAP w/o ECR, which removes the editing coverage rate from the reward function. The results are demonstrated in Table 12 and Table 13. Specifically, we have the following observations. LEAP performs superior in most scenarios. We attribute this to learning different prompts for each node being inherently more challenging than learning k basic universal graph prompt vectors and k linear projections. On the other hand, LEAP w/o ECR demonstrates the least satisfactory performance in the full-shot scenario. We ascribe this to the absence of the editing convergence rate constraint, which leads the model to repeatedly select and modify a small subset of nodes.

C.3 Hyper-parameter Analysis

We analyze the critical hyper-parameters in LEAP, including the basic universal graph prompt vector number k, the prompt edit range ϑ, the policy networks update interval h, and the policy finite horizon (step) T. For both graph- and node-level tasks, we report the average performance across all datasets for each pre-training strategy.

C.3.1 Basic Universal Graph Prompt Vector Number k. From Figure 3 and Figure 4, we observe that the optimal value of the number of basic universal graph prompt vectors k is 10. Moreover, selecting k as 5 or 20 has no significant impact on performance, demonstrating LEAP's robustness to hyper-parameters k.

C.3.2 Prompt Edit Range ϑ. From Figure 5 and Figure 6, we observe that the optimal value of prompt edit range ϑ is 0.5. Moreover, varying ϑ within reasonable limits has minimal impact on the model's performance, highlighting LEAP's robustness to the hyper-parameter ϑ.

C.3.3 Policy Networks Update Interval h. From Figure 7 and Figure 8, we observe that the optimal value of policy network update interval h is 3 for the graph-level task and 4 for the node-level task. Moreover, varying h within reasonable limits has minimal impact on the model's performance, showing LEAP's robustness to the hyper-parameter h.

C.3.4 Policy Finite Horizon (Step) T. From Figure 9 and Figure 10, we observe that the optimal value of policy finite horizon (step) T is $N/4$ in the full-shot scenario and $N/2$ in the few-shot scenario. Moreover, varying T within reasonable limits has little effect on the model's performance, demonstrating LEAP's robustness to the hyper-parameter T.

D Discussion

We categorize previous works [3, 12, 42] into two paradigms of universal graph prompt tuning. Specifically, GPF and GPF-plus fall under the *Conventional Graph Prompt Tuning* paradigm, which ensures that the universal graph can theoretically achieve the equivalent effect of any prompting function. However, this paradigm lacks a tailored design for pursuing more ideal prompts. On the other hand, SUPT and RELIEF belong to the *Selective Node-based Graph Prompt Tuning* paradigm, which proposes various tailored designs to select and generate prompts for some nodes. However, as stated in Theorem 1, it compromises the theoretical foundation of universal graph prompt tuning.

Our proposed LEAP represents an ideal paradigm, which we refer to as the *Learning and Editing Graph Prompt Tuning* paradigm. It preserves the theoretical foundation of universal graph prompt tuning by constructing the basic universal graph prompt, while simultaneously pursuing more ideal prompts through prompt editing, effectively addressing the limitations of the previous two paradigms.

In the following, we will analyze the connections between LEAP and these previous works in detail.

D.1 Compare with GPF and GPF-plus

Compared to GPF and GPF-plus, LEAP builds upon the construction of a basic universal graph prompt to ensure a consistent theoretical foundation. Beyond this, LEAP further leverages RL to select and edit prompts for some nodes to pursue more ideal prompts. By optimizing prompts through maximizing performance gains and edit convergence rates, LEAP achieves practical performance that more closely approaches the theoretical upper bound.

D.2 Compare with SUPT

SUPT generates subgraph-level prompts to learn higher-quality prompts related to the context. However, the prompt is applied only to a subgraph rather than the entire node set of a given graph, which undermines the theoretical foundation of universal graph prompt tuning. In contrast, LEAP ensures the theoretical foundation of universal graph prompt tuning while leveraging RL to edit prompts for higher cumulative rewards.

D.3 Compare with RELIEF

RELIEF is inspired by NLP and utilizes RL to select and add prompts for certain nodes. However, as stated in Theorem 1, this approach undermines the theoretical foundation of universal graph prompt

Table 10: ROC-AUC (%) and standard deviation for graph classification on molecule property prediction benchmark under 50-shot scenario with various pre-training and tuning strategies. The best results for each dataset and pre-training strategy are bolded, while the runners-up are underlined.

	Tuning Strategy	BBBP	Tox21	ToxCast	SIDER	ClinTox	MUV	HIV	BACE	**Avg.**
Infomax	FT	65.03 ±1.13	71.41 ±0.69	58.04 ±0.36	53.47 ±0.48	61.11 ±2.40	65.20 ±0.45	65.87 ±2.41	65.65 ±0.89	63.22
	GPF	66.85 ±1.14	71.19 ±0.35	58.38 ±0.20	53.66 ±0.45	65.00 ±2.75	67.17 ±0.13	66.51 ±1.91	65.59 ±1.29	64.29
	GPF-plus	66.94 ±1.70	71.50 ±0.32	58.55 ±0.33	53.89 ±0.72	65.85 ±1.99	67.22 ±0.11	66.91 ±1.44	66.23 ±1.95	64.64
	$SUPT_{soft}$	66.21 ±0.91	71.40 ±0.26	58.73 ±0.29	53.49 ±0.49	65.82 ±3.05	66.69 ±0.31	66.59 ±1.88	66.44 ±2.01	64.42
	$SUPT_{hard}$	66.00 ±1.02	71.23 ±0.40	58.60 ±0.26	53.73 ±0.40	65.83 ±2.55	66.75 ±1.19	66.81 ±2.26	66.80 ±0.75	64.47
	RELIEF	67.21 ±1.35	71.51 ±0.28	58.70 ±0.23	53.81 ±0.47	66.20 ±2.15	67.09 ±0.16	67.04 ±1.51	**67.56** ±0.81	64.89
	LEAP	**68.27** ±1.09	**71.85** ±0.21	**58.79** ±0.22	**54.00** ±0.56	**66.44** ±2.29	**67.28** ±0.49	**67.15** ±1.38	67.31 ±0.91	**65.14**
AttrMasking	FT	66.26 ±0.69	72.36 ±0.28	57.50 ±0.36	55.24 ±0.58	64.55 ±3.56	62.09 ±1.01	58.96 ±1.39	66.86 ±1.50	62.98
	GPF	66.41 ±1.03	72.55 ±0.24	58.15 ±0.22	55.61 ±1.27	69.52 ±2.72	62.70 ±0.30	58.80 ±1.50	68.26 ±1.14	64.00
	GPF-plus	66.83 ±0.89	72.71 ±0.45	58.21 ±0.30	55.98 ±0.99	70.94 ±2.01	62.99 ±0.37	59.13 ±1.31	68.90 ±1.00	64.46
	$SUPT_{soft}$	66.33 ±1.17	72.63 ±0.48	58.24 ±0.40	55.79 ±1.11	68.89 ±3.15	62.85 ±0.20	58.98 ±1.26	67.05 ±1.61	63.85
	$SUPT_{hard}$	66.80 ±1.20	72.74 ±0.30	58.19 ±0.35	55.88 ±1.03	69.41 ±2.09	62.94 ±0.27	59.11 ±1.10	68.98 ±0.79	64.26
	RELIEF	66.86 ±0.61	72.44 ±0.38	**58.43** ±0.27	56.04 ±0.85	72.79 ±1.99	63.09 ±0.14	**59.40** ±0.62	69.37 ±0.70	64.80
	LEAP	**66.95** ±0.35	**72.79** ±0.22	**58.43** ±0.20	**56.17** ±0.91	**73.33** ±2.22	**63.42** ±0.20	59.33 ±0.82	**69.45** ±0.88	**64.98**
ContextPred	FT	62.69 ±0.97	70.28 ±0.46	57.90 ±0.44	56.33 ±0.70	61.27 ±2.53	63.30 ±1.07	59.35 ±1.78	66.74 ±2.01	62.23
	GPF	61.77 ±1.03	70.42 ±0.33	58.48 ±0.24	56.37 ±0.40	60.85 ±2.33	64.28 ±0.38	59.64 ±2.00	68.42 ±1.37	62.53
	GPF-plus	62.12 ±1.30	70.48 ±0.37	58.34 ±0.26	56.79 ±0.39	62.03 ±2.40	64.59 ±0.42	59.41 ±1.88	69.18 ±1.05	62.87
	$SUPT_{soft}$	61.93 ±1.34	70.40 ±0.15	57.91 ±0.39	56.41 ±0.57	59.50 ±2.13	64.42 ±1.13	59.31 ±1.55	68.75 ±1.30	62.33
	$SUPT_{hard}$	62.11 ±1.19	70.31 ±0.20	58.15 ±0.23	56.31 ±0.49	59.72 ±2.39	61.19 ±0.89	59.28 ±1.35	69.64 ±1.22	62.09
	RELIEF	62.86 ±0.90	70.56 ±0.25	58.32 ±0.24	56.73 ±0.39	**62.66** ±1.39	64.37 ±0.80	59.64 ±1.92	70.07 ±0.90	63.15
	LEAP	**62.98** ±0.81	**70.68** ±0.17	**58.53** ±0.35	**56.96** ±0.58	62.29 ±1.65	**64.92** ±0.61	**59.99** ±1.58	**70.33** ±0.84	**63.34**
GCL	FT	62.81 ±1.01	61.31 ±0.67	54.04 ±0.77	51.66 ±0.64	75.35 ±2.18	52.01 ±2.06	59.10 ±1.50	51.00 ±3.21	58.41
	GPF	61.86 ±1.64	61.09 ±0.88	54.40 ±0.37	51.55 ±0.50	78.91 ±2.22	52.32 ±1.69	60.02 ±1.30	58.09 ±1.77	59.78
	GPF-plus	62.99 ±1.63	61.28 ±0.49	54.55 ±0.32	51.95 ±0.95	79.16 ±1.91	52.70 ±1.95	60.21 ±1.45	60.61 ±2.56	60.43
	$SUPT_{soft}$	62.64 ±0.98	**61.65** ±0.20	54.50 ±0.56	51.49 ±1.32	78.28 ±2.50	52.52 ±1.72	59.75 ±1.79	59.55 ±2.39	60.05
	$SUPT_{hard}$	62.55 ±1.52	61.52 ±0.29	54.23 ±0.51	51.81 ±0.80	78.34 ±2.21	51.99 ±1.45	59.96 ±2.04	60.10 ±2.10	59.95
	RELIEF	**63.73** ±0.74	61.49 ±0.68	54.73 ±0.30	52.20 ±0.70	79.53 ±1.85	53.55 ±1.80	60.40 ±2.12	62.47 ±1.30	61.01
	LEAP	63.55 ±1.08	61.54 ±0.39	**54.89** ±0.35	**52.33** ±1.01	**79.93** ±1.96	**53.59** ±1.91	**60.53** ±1.98	**62.50** ±1.41	**61.11**

Table 11: Accuracy (%) and Macro F1-score (%) with respective standard deviation for node classification under 10-shot scenario with two pre-training and various tuning strategies.

	Tuning Strategy	Cora		CiteSeer		PubMed		Computers		Photos	
		Accuracy	Macro F1	Accuracy	Macro F1	Accuracy	Macro F1	Accuracy	Macro F1	Accuracy	Macro F1
MaskedEdge	FT	54.21 ±1.80	53.30 ±1.65	60.83 ±1.35	56.25 ±1.23	70.91 ±1.89	65.42 ±1.73	74.61 ±1.80	70.52 ±1.73	83.52 ±2.02	81.19 ±2.13
	GPPT	47.62 ±0.74	46.99 ±0.60	57.33 ±0.82	51.49 ±0.86	70.11 ±0.92	65.05 ±0.88	73.80 ±0.81	68.64 ±0.77	82.05 ±0.94	80.41 ±1.11
	GPF	55.35 ±1.52	54.28 ±1.31	60.32 ±1.17	55.55 ±1.30	69.81 ±2.90	64.97 ±2.41	73.29 ±2.06	68.03 ±1.95	83.08 ±0.92	80.93 ±0.77
	GPF-plus	56.72 ±1.26	55.03 ±1.03	63.36 ±0.91	56.92 ±0.75	**72.08** ±2.31	66.18 ±2.01	75.22 ±1.27	**72.02** ±1.35	84.82 ±1.30	82.61 ±1.04
	$SUPT_{soft}$	55.31 ±1.71	54.29 ±1.66	60.02 ±1.99	54.99 ±1.58	71.18 ±2.15	65.90 ±2.03	72.28 ±2.91	67.91 ±2.66	84.00 ±1.25	82.07 ±1.69
	$SUPT_{hard}$	56.31 ±1.08	55.45 ±1.37	61.88 ±1.43	56.73 ±1.25	70.59 ±1.86	65.29 ±1.99	72.56 ±1.31	68.20 ±1.28	84.33 ±0.88	82.45 ±0.72
	RELIEF	57.03 ±1.21	55.39 ±0.99	63.33 ±1.40	57.51 ±1.05	71.16 ±1.68	66.37 ±1.21	73.02 ±1.33	69.97 ±1.60	84.73 ±1.29	83.24 ±0.88
	LEAP	**58.32** ±1.50	**56.81** ±0.94	**64.24** ±1.56	**57.67** ±1.54	71.68 ±1.48	**66.65** ±1.61	**75.48** ±1.33	71.09 ±1.09	**85.19** ±1.50	**83.35** ±1.42
ContraEdge	FT	60.28 ±1.99	60.09 ±1.82	62.61 ±2.09	56.48 ±2.15	70.15 ±1.44	65.84 ±1.51	80.02 ±1.03	76.25 ±1.21	85.02 ±1.45	81.97 ±1.58
	GPrompt	56.98 ±2.53	54.55 ±2.48	59.30 ±0.81	55.88 ±0.94	69.50 ±0.84	64.73 ±0.77	69.81 ±1.92	67.13 ±1.90	76.30 ±2.23	73.58 ±2.17
	GPF	59.31 ±1.42	58.84 ±1.58	63.37 ±2.01	55.79 ±2.04	70.91 ±2.21	65.99 ±1.88	80.20 ±1.18	76.91 ±1.29	85.39 ±1.55	82.64 ±1.75
	GPF-plus	60.93 ±1.04	60.40 ±1.10	65.03 ±1.89	57.14 ±1.72	71.18 ±3.73	66.39 ±3.67	80.85 ±1.03	77.05 ±1.12	86.12 ±1.30	83.43 ±1.61
	$SUPT_{soft}$	60.03 ±1.44	59.61 ±1.20	63.69 ±2.01	56.57 ±2.19	70.99 ±3.15	66.08 ±3.03	81.03 ±1.15	76.31 ±0.94	85.15 ±1.54	82.33 ±1.09
	$SUPT_{hard}$	59.37 ±0.96	59.09 ±0.89	64.50 ±1.89	56.26 ±1.94	70.86 ±2.27	66.23 ±2.30	80.20 ±1.04	76.43 ±1.27	84.68 ±1.35	82.12 ±1.10
	RELIEF	60.93 ±1.40	60.77 ±1.08	**66.11** ±1.01	57.71 ±0.92	71.35 ±1.58	66.82 ±1.49	80.98 ±1.33	77.85 ±1.51	86.26 ±1.10	83.61 ±0.87
	LEAP	**61.33** ±0.92	**61.12** ±1.14	65.92 ±1.01	**57.90** ±0.85	**71.84** ±1.14	**66.95** ±1.21	**81.32** ±1.55	**78.21** ±1.60	**86.41** ±1.39	**83.95** ±1.32

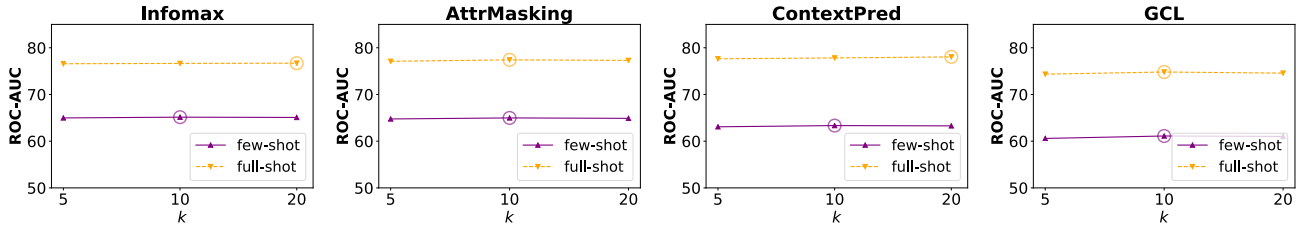

Figure 3: The average ROC-AUC across all datasets for the graph-level task under different hyper-parameter k, with the circle marking the optimal results.

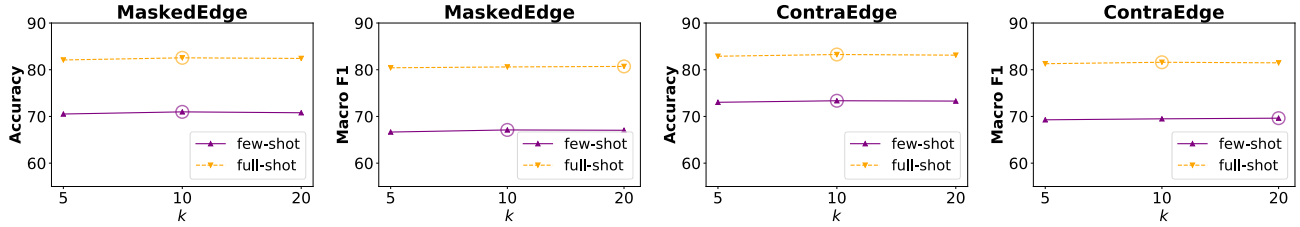

Figure 4: The average Accuracy and Macro F1 across all datasets for the node-level task under different hyper-parameter k, with the circle marking the optimal results.

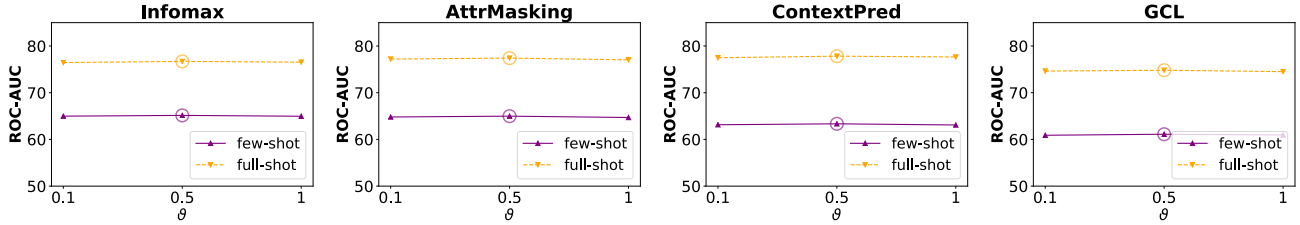

Figure 5: The average ROC-AUC across all datasets for the graph-level task under different hyper-parameter ϑ, with the circle marking the optimal results.

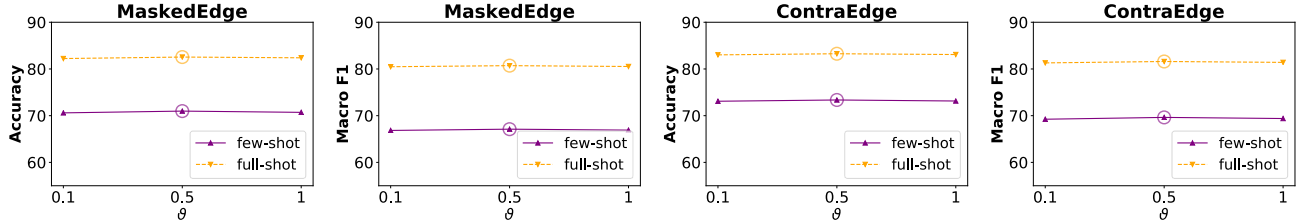

Figure 6: The average Accuracy and Macro F1 across all datasets for the node-level task under different hyper-parameter ϑ, with the circle marking the optimal results.

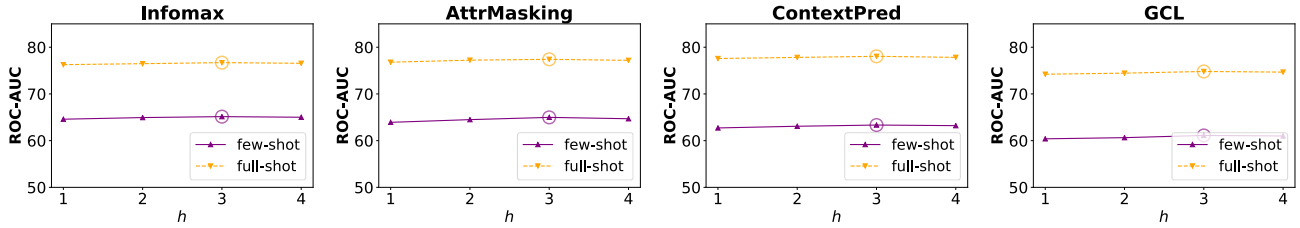

Figure 7: The average ROC-AUC across all datasets for the graph-level task under different hyper-parameter h, with the circle marking the optimal results.

Table 12: ROC-AUC (%) and standard deviation for graph classification on molecule property prediction benchmark under both full-shot and few-shot scenarios with various pre-training and variants.

Datasets	Variants	Infomax		AttrMasking		ContextPred		GCL	
		full-shot	few-shot	full-shot	few-shot	full-shot	few-shot	full-shot	few-shot
ToxCast	LEAP	**67.99**	**58.79**	67.83	**58.43**	**68.46**	**58.53**	**64.92**	**54.89**
	LEAP_{GPF}	67.70	58.40	67.39	58.19	68.31	58.30	64.73	54.50
	$\text{LEAP}_{\text{GPF-plus}}$	67.94	58.52	**67.84**	58.32	**68.46**	58.42	64.86	54.67
	LEAP w/o ECR	67.32	58.47	67.42	58.28	68.10	58.34	64.70	54.48
ClinTox	LEAP	**75.98**	**66.44**	**77.27**	**73.33**	**77.04**	**62.29**	**80.08**	**79.93**
	LEAP_{GPF}	75.39	65.82	77.15	71.56	76.81	61.98	79.23	79.18
	$\text{LEAP}_{\text{GPF-plus}}$	75.87	65.99	77.24	72.91	76.99	62.22	80.01	79.58
	LEAP w/o ECR	75.36	65.80	77.00	72.46	76.44	62.05	79.23	79.32
MUV	LEAP	81.49	**67.28**	**80.80**	**63.42**	**84.21**	**64.92**	**74.28**	**53.59**
	LEAP_{GPF}	81.28	67.20	80.65	62.94	84.02	64.49	74.01	52.80
	$\text{LEAP}_{\text{GPF-plus}}$	**81.62**	67.22	80.75	63.19	84.15	64.62	74.26	53.25
	LEAP w/o ECR	80.83	67.15	80.33	62.90	83.98	64.40	73.79	53.21
BACE	LEAP	84.35	**67.31**	**86.16**	**69.45**	**86.30**	**70.33**	**78.34**	**62.50**
	LEAP_{GPF}	84.16	66.09	85.85	68.58	85.77	69.20	78.04	60.71
	$\text{LEAP}_{\text{GPF-plus}}$	**84.37**	67.01	86.06	69.22	86.03	69.78	78.30	61.50
	LEAP w/o ECR	84.08	67.08	85.47	68.98	85.42	70.00	77.35	62.12

Table 13: Accuracy (%) and Macro F1-score (%) with respective standard deviation for node classification under both full-shot and few-shot scenarios with two pre-training and various variants.

Datasets	Variants	MaskedEdge				ContraEdge			
		full-shot		few-shot		full-shot		few-shot	
		Accuracy	Macro F1	Accuracy	Macro F1	Accuracy	Macro F1	Accuracy	Macro F1
Cora	LEAP	**82.81**	**80.77**	**58.32**	**56.81**	**81.41**	**80.70**	**61.33**	**61.12**
	LEAP_{GPF}	81.90	79.46	56.32	55.20	80.75	79.81	59.57	59.02
	$\text{LEAP}_{\text{GPF-plus}}$	82.68	80.50	57.39	55.72	81.11	80.37	61.01	60.54
	LEAP w/o ECR	80.97	78.78	56.65	55.53	80.38	80.04	60.88	60.37
CiteSeer	LEAP	**71.54**	**69.09**	**64.24**	**57.67**	**73.81**	**71.44**	**65.92**	**57.90**
	LEAP_{GPF}	70.80	68.56	62.02	56.66	73.12	70.65	64.00	56.49
	$\text{LEAP}_{\text{GPF-plus}}$	71.39	68.99	64.10	57.59	73.72	71.32	65.45	57.40
	LEAP w/o ECR	70.45	68.09	63.02	57.26	72.66	70.30	65.45	57.28
Photos	LEAP	**91.04**	**89.63**	**85.19**	**83.35**	**91.67**	**90.26**	**86.41**	**83.95**
	LEAP_{GPF}	90.30	89.06	83.48	81.20	91.35	89.89	85.66	82.84
	$\text{LEAP}_{\text{GPF-plus}}$	90.90	89.51	84.67	82.75	91.60	90.18	86.18	83.53
	LEAP w/o ECR	90.19	88.99	84.56	82.71	91.00	89.63	86.10	83.46

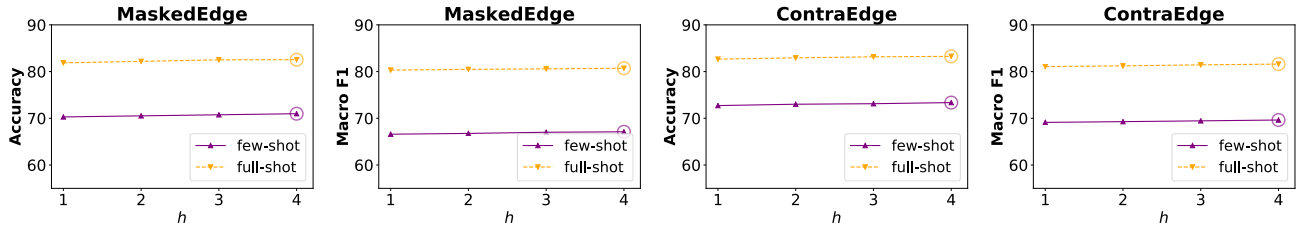

Figure 8: The average Accuracy and Macro F1 across all datasets for the node-level task under different hyper-parameter h, with the circle marking the optimal results.

tuning in the graph learning domain. Furthermore, as analyzed in Section 5, RELIEF performs well in few-shot scenarios but exhibits mediocre performance in full-shot settings. To mitigate the overfitting caused by RL, RELIEF employs ensemble actor policy networks to enhance generalization, which inevitably introduces

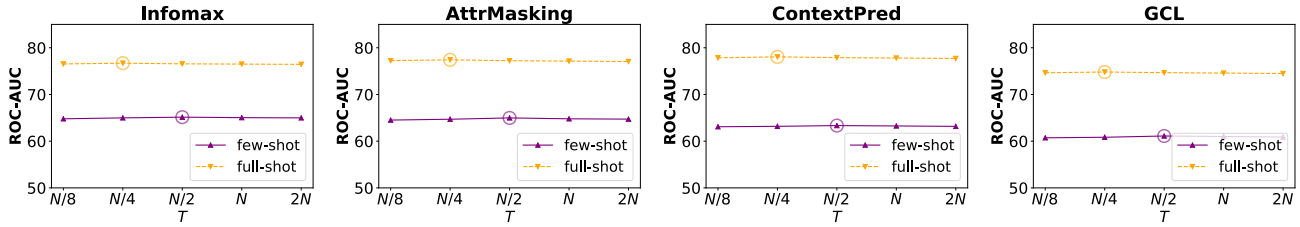

Figure 9: The average ROC-AUC across all datasets for the graph-level task under different hyper-parameter T, with the circle marking the optimal results.

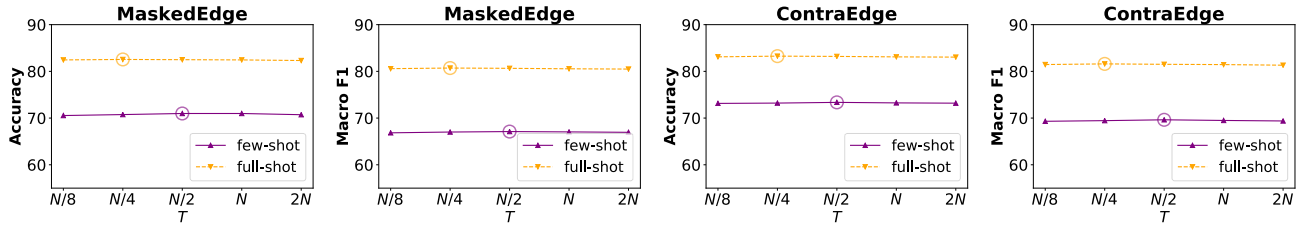

Figure 10: The average Accuracy and Macro F1 across all datasets for the node-level task under different hyper-parameter T, with the circle marking the optimal results.

additional computational overhead. In contrast, LEAP's basic universal graph prompt ensures the theoretical foundation of universal graph prompt tuning. Moreover, LEAP introduces a carefully designed reward function that addresses the issue of repeatedly selecting a small subset of nodes through the edit convergence rate. Additionally, unlike RELIEF, LEAP eliminates the need for ensemble actor policy networks to address overfitting, as its basic universal graph prompt and RL are naturally aligned to optimize a shared objective.

E Complexity Analysis

We analyze the complexity of LEAP for one epoch. We list definitions used in our analysis. Specifically, let L_1 and L_2 denote the layer numbers of the pre-trained graph model and policy networks, respectively. Moreover, the hidden dimensions of the pre-trained graph model and policy networks are defined as d_1 and d_2, respectively. The projection head is a L_3 layer MLP with hidden size d_3. N and E represent the maximum number of nodes and edges, respectively. D is the dimension of node features. m is the total number of graphs.

E.1 Prompted Graph Building

We summarize basic universal graph prompt building and RL-driven prompt editing as prompted graph building. First, we build the basic universal graph prompt via k basic vectors and k linear projections, with a complexity of $O(k^2N)$. Furthermore, for each step t, we get f_a via both discrete and continuous actors, with a complexity of $O(2(2d_1d_2 + L_2d_2^2))$. Notably, we adopt T steps within each epoch. Therefore, the total time complexity is:

$$m \cdot (O(k^2N) + T \cdot O(4d_1d_2 + 2L_2d_2^2)). \tag{12}$$

E.2 Policy Networks Training

A total of T steps are utilized for sequentially updating the actors and the critic, which primarily involves the training of policy networks based on MLP updates. Notably, the policy networks are updated at intervals of h, thereby reducing the per-epoch complexity by a factor of h. Therefore, the total time complexity is:

$$mT \cdot O(3(2d_1d_2 + L_2d_2^2))/h. \tag{13}$$

E.3 Projection Head and Basic Prompt Training

In this process, we adopt pre-trained graph model to obtain node representations, with a complexity of $O(L_1(E + ND)d_1)$. Moreover, we feed representations into the projection head, with a complexity of $O(2d_1d_3 + L_3d_3^2)$. Final loss is calculated by label and output from the projection head, with a complexity of $O(1)$. Therefore, the total time complexity is:

$$m \cdot (O(L_1(E + ND)d_1) + O(2d_1d_3 + L_3d_3^2) + O(1)). \tag{14}$$

E.4 Simplified Complexity

For simplification, we adopt a unified dimension d for all d_1, d_2, and d_3. Moreover, we adopt a unified layer L for all L_1, L_2, and L_3. Therefore, the simplified total time complexity is:

$$m \cdot (O(k^2N + Ld(E + ND) + 1) + (\frac{3T}{h} + 2T + 1) \cdot O((2 + L)d^2)). \tag{15}$$

In practice, we set $T = N/4$ and $T = N/2$ in the full-shot and few-shot scenarios, respectively. Therefore, in the full-shot scenario, the simplified total time complexity is:

$$m \cdot (O(k^2N + Ld(E + ND) + 1) + (\frac{3N}{4h} + \frac{N}{2} + 1) \cdot O((2 + L)d^2)). \tag{16}$$

Moreover, in the few-shot scenario, the simplified total time complexity is:

$$m \cdot (O(k^2N + Ld(E + ND) + 1) + (\frac{3N}{2h} + N + 1) \cdot O((2 + L)d^2)). \tag{17}$$

These two simplified total time complexities indicate that the computation time will noticeably increase with the number of large graphs as they scale up.

F Related Work

F.1 Graph Prompt Tuning

Task-specific Graph Prompt Tuning Task-specific approaches aim to align pretext tasks with downstream objectives. For instance, GPPT [24] and GraphPrompt [18] utilize edge prediction to bridge pre-training and downstream tasks, while All in One [26] employs a prompt graph for graph-level contrastive learning. Similarly, SGL-PT [43] connects downstream tasks with masked node prediction via contrastive and generative objectives. However, these approaches may face challenges when GNNs are pre-trained with diverse self-supervised tasks through multi-task learning, rather than simpler objectives like link prediction.

Universal Graph Prompt Tuning To address the limitations of task-specific methods, universal graph prompt tuning approaches have been introduced, offering compatibility across various pre-training strategies. GPF [3] pioneered this direction by demonstrating that adding a learnable, uniform feature prompt vector to each node is theoretically equivalent to any prompting function, irrespective of the pre-training strategy. GPF-plus [3] further extended this idea by introducing node-specific prompted features for more fine-grained and adaptable tuning. Recent works have explored selective node-based graph prompt tuning to improve optimization. For example, SUPT [12] assigns feature prompts at the subgraph level, capturing nuanced contextual information and enhancing performance. RELIEF [42] takes a reinforcement learning approach to dynamically select nodes for prompt addition, enabling high-quality and adaptive prompt tuning. Although these selective node-based graph prompt tuning approaches achieve certain performance improvements, we point out that selective node-based graph prompt tuning undermines the theoretical foundation. This limitation constrains the theoretical representational capacity of universal graph prompt tuning.

F.2 RL for Graph Learning

The integration of RL into graph representation learning has seen notable progress. MAG-GNN [11] leverages DQN to select optimal subgraph subsets, enhancing graph expressivity. WSD [29] employs weighted sampling for subgraph counting, with edge weights determined using DDPG. SUGAR [25] maintains hierarchical graph properties by selecting subgraphs via an RL-based pooling mechanism grounded in Q-learning. GPA [7] utilizes DQN to learn optimal annotation strategies for identifying valuable nodes in active search, while GraphCBAL [41] extends this by employing A2C for a class-balanced active search strategy. Additionally, RELIEF [42] incorporates an RL algorithm specifically designed for hybrid action spaces, integrating policy generalization techniques to select subsets of nodes and effectively incorporate prompts. However, RL inevitably introduces extra computational overhead to graph learning. To address these limitations, we propose the Learning and Editing Universal Prompt Tuning paradigm, which combines basic universal graph prompt learning with RL-driven editing to optimize prompts while maintaining a solid theoretical foundation and incurring affordable computational overhead.

G Proofs

G.1 Proof of Theorem 1

Proof Sketch The proof consists of two parts:

- **Sufficiency**: If every node is assigned a prompt, universal graph prompt tuning can simulate any prompting function $\psi(\cdot)$.
- **Necessity**: If any node lacks a prompt, there exists at least one $\psi(\cdot)$ that cannot be simulated.

G.1.1 Sufficiency. Assume each node v_i is assigned a prompt p_i. Following the proof of the Theorem in the GPF, any prompting function $\psi(\cdot)$ can be decomposed into three atomic operations:

- **Feature Modification:** Modifying node features $\mathbf{X} \to \hat{\mathbf{X}}$.
- **Structure Modification:** Modifying adjacency matrix $\mathbf{A} \to \hat{\mathbf{A}}$.
- **Component Modification:** Adding or removing isolated components (sub-graphs) and generating the new adjacency matrix and feature matrix $(\mathbf{X}, \mathbf{A}) \to (\hat{\mathbf{X}}, \hat{\mathbf{A}})$.

Simulate each operation via universal graph prompts (For analytical simplicity, we initially assume that f_θ is a single-layer GNN with a linear transformation. Subsequently, we extend our conclusions to multi-layer models utilizing various transition matrices):

Case 1: Feature Modification:

Set $p_i = \Delta\mathbf{X}_{[i,:]}$ for all i. Then:

$$\mathbf{X} + \{p_1, \ldots, p_N\} = \mathbf{X} + \Delta\mathbf{X} = \hat{\mathbf{X}}. \tag{18}$$

This trivially replicates any prompting function $\psi(\cdot)$.

Case 2: Structure Modification:

Structural changes in $\mathbf{A}$ affect the diffusion matrix [4] $\mathbf{S} = \mathbf{A} + (1+\epsilon)\mathbf{I}$. Let $\hat{\mathbf{S}} = \hat{\mathbf{A}} + (1+\epsilon)\mathbf{I}$. The output of f_θ under $\hat{\mathbf{A}}$ is:

$$f_\theta(\hat{\mathbf{A}}, \mathbf{X}) = \hat{\mathbf{S}}\mathbf{X}\mathbf{W} = \mathbf{S}\mathbf{X}\mathbf{W} + \Delta\mathbf{S}\mathbf{X}\mathbf{W}, \quad \Delta\mathbf{S} = \hat{\mathbf{S}} - \mathbf{S}, \tag{19}$$

where $\mathbf{W}$ is a frozen linear transformation. The parameters ϵ and $\mathbf{W}$ have been pre-trained in advance and remain fixed during downstream tuning. To simulate this with prompts, solve:

$$\mathbf{S}(\mathbf{X} + p)\mathbf{W} = \mathbf{S}\mathbf{X}\mathbf{W} + \Delta\mathbf{S}\mathbf{X}\mathbf{W}. \tag{20}$$

Simplify to:

$$\mathbf{S}p\mathbf{W} = \Delta\mathbf{S}\mathbf{X}\mathbf{W}. \tag{21}$$

Since $\mathbf{W}$ is full-rank (pre-trained), we require:

$$\mathbf{S}p = \Delta\mathbf{S}\mathbf{X}. \tag{22}$$

Let $p = \{p_1, \ldots, p_N\}$. For each node i:

$$\sum_j \mathbf{S}_{[i,j]} p_j = \sum_j \Delta\mathbf{S}_{[i,j]} \mathbf{X}_{[j,:]}. \tag{23}$$

This is a linear system with $N \times D$ variables (p_j) and $N \times D$ equations. A solution exists if the system is consistent.

Case 3: Component Modification:

Suppose $\psi(\cdot)$ adds a disconnected subgraph $C = (\mathbf{A}_c, \mathbf{X}_c)$. Let $\hat{\mathbf{A}} = \begin{bmatrix} \mathbf{A} & 0 \\ 0 & \mathbf{A}_c \end{bmatrix}, \hat{\mathbf{X}} = \begin{bmatrix} \mathbf{X} \\ \mathbf{X}_c \end{bmatrix}$. The output of f_θ is:

$$f_\theta(\mathbf{A}, \mathbf{X}) = \mathbf{S}\mathbf{X}\mathbf{W}, \quad \mathbf{S} = \mathbf{A} + (1+\epsilon)\mathbf{I}. \tag{24}$$

The target output after adding C is:

$$f_\theta(\hat{\mathbf{A}}, \hat{\mathbf{X}}) = \hat{\mathbf{S}}\hat{\mathbf{X}}\mathbf{W}, \quad \hat{\mathbf{S}} = \begin{bmatrix} \mathbf{S} & 0 \\ 0 & \mathbf{S}_c \end{bmatrix}, \quad \mathbf{S}_c = \mathbf{A}_c + (1+\epsilon)\mathbf{I}. \tag{25}$$

To replicate this with prompts, we require:

$$\mathbf{S}(\mathbf{X} + p)\mathbf{W} = \hat{\mathbf{S}}\hat{\mathbf{X}}\mathbf{W}. \tag{26}$$

Cancel $\mathbf{W}$ (full-rank) and expand both sides:

$$\mathbf{S}\mathbf{X} + \mathbf{S}p = \begin{bmatrix} \mathbf{S}\mathbf{X} \\ \mathbf{S}_c\mathbf{X}_c \end{bmatrix}. \tag{27}$$

This implies:

$$\mathbf{S}p = \begin{bmatrix} 0 \\ \mathbf{S}_c\mathbf{X}_c \end{bmatrix}. \tag{28}$$

For existing nodes $v_i \in \mathcal{V}$:

$$\sum_j \mathbf{s}_{[i,j]} p_j = 0 \quad \Longrightarrow \quad p_j = 0 \quad \text{(since } \mathbf{S} \text{ is invertible)}. \tag{29}$$

For virtual nodes $v_j \in C$. To simulate $\mathbf{S}_c\mathbf{X}_c$, define prompts for virtual nodes as:

$$p_j = \mathbf{X}_{c[j,:]} \quad (\text{ for } j = N+1, \ldots, N+M), \tag{30}$$

where M denotes the number of virtual nodes $v_j \in C$. This is a system of $M \times D$ equations. A solution exists if the prompt dimension D is sufficiently large (e.g., $D \geq M$).

Extension to Multi-Layer Scenario

The final node representation after L layers is:

$$\mathbf{H}_{(L)} = \underbrace{\mathbf{S}_{(L)}\mathbf{S}_{(L-1)}\cdots\mathbf{S}_{(1)}}_{\mathbf{S}'}\mathbf{X}\underbrace{\mathbf{W}_{(1)}\mathbf{W}_{(2)}\cdots\mathbf{W}_{(L)}}_{\mathbf{W}'}. \tag{31}$$

Let $\mathbf{S}' = \prod_{l=1}^{L}\mathbf{S}_{(l)}$ and $\mathbf{W}' = \prod_{l=1}^{L}\mathbf{W}_{(l)}$. The output simplifies to:

$$\mathbf{H}_{(L)} = \mathbf{S}'\mathbf{X}\mathbf{W}'. \tag{32}$$

Case 1: Feature Modification:

Let $\hat{\mathbf{X}} = \mathbf{X} + \Delta\mathbf{X}$. The target output is:

$$\hat{\mathbf{H}}_{(L)} = \mathbf{S}'\hat{\mathbf{X}}\mathbf{W}' = \mathbf{S}'(\mathbf{X} + \Delta\mathbf{X})\mathbf{W}'. \tag{33}$$

Achieving equivalence via prompts needs to satisfy the following:

$$\mathbf{S}'(\mathbf{X} + p)\mathbf{W}' = \mathbf{S}'(\mathbf{X} + \Delta\mathbf{X})\mathbf{W}'. \tag{34}$$

We require:

$$\mathbf{S}'p = \mathbf{S}'\Delta\mathbf{X}. \tag{35}$$

Since $\mathbf{S}'$ is invertible (diagonally dominant), the solution is:

$$p = \Delta\mathbf{X}. \tag{36}$$

Thus, setting $p_i = \Delta\mathbf{X}_{[i,:]}$ for all i replicates the feature modification.

Case 2: Structure Modification:

Let $\mathbf{S}'_{(l)} = \hat{\mathbf{A}} + (1+\epsilon_l)\mathbf{I}$. The target output is:

$$\mathbf{H}'_{(L)} = \left(\prod_{l=1}^{N}\mathbf{S}'_{(l)}\right)\mathbf{X}\mathbf{W}' \triangleq \mathbf{S}''\mathbf{X}\mathbf{W}'. \tag{37}$$

To replicate $\mathbf{H}'_{(L)}$, solve:

$$\mathbf{S}'(\mathbf{X} + p)\mathbf{W}' = \mathbf{S}''\mathbf{X}\mathbf{W}'. \tag{38}$$

Canceling $\mathbf{W}'$:

$$\mathbf{S}'p = (\mathbf{S}'' - \mathbf{S}')\mathbf{X} \tag{39}$$

For each node i:

$$\sum_j \mathbf{S}'_{[i,j]} p_j = \sum_j (\mathbf{S}''_{[i,j]} - \mathbf{S}'_{[i,j]})\mathbf{X}_{[j,:]}. \tag{40}$$

This linear system has a unique solution p due to $\mathbf{S}''$'s invertibility.

Case 3: Component Modification:

Suppose $\psi(\cdot)$ adds a disconnected subgraph $C = (\mathbf{A}_c, \mathbf{X}_c)$. Let $\hat{\mathbf{A}} = \begin{bmatrix} \mathbf{A} & 0 \\ 0 & \mathbf{A}_c \end{bmatrix}, \hat{\mathbf{X}} = \begin{bmatrix} \mathbf{X} \\ \mathbf{X}_c \end{bmatrix}$. The output is:

$$\hat{\mathbf{H}}_{(L)} = \hat{\mathbf{S}}'\hat{\mathbf{X}}\mathbf{W}' = \begin{bmatrix} \mathbf{S}'\mathbf{X} \\ \mathbf{S}'_c\mathbf{X} \end{bmatrix}\mathbf{W}', \tag{41}$$

where $\mathbf{S}'_c = \prod_{l=1}^{L}(\mathbf{A}_c + (1+\epsilon_l)\,\mathbf{I})$. To simulate $\hat{\mathbf{H}}_{(L)}$, enforce:

$$\mathbf{S}'(\mathbf{X} + p)\mathbf{W}' = \begin{bmatrix} \mathbf{S}'\mathbf{X} \\ \mathbf{S}'_c\mathbf{X}_c \end{bmatrix}\mathbf{W}'. \tag{42}$$

Canceling $\mathbf{W}'$:

$$\mathbf{S}'p = \begin{bmatrix} 0 \\ \mathbf{S}'_c\mathbf{X}_c \end{bmatrix}. \tag{43}$$

Therefore, for existing nodes $v_i \in \mathcal{V}$, set $p_i = 0$. For virtual nodes $v_j \in C$, inject their impact into existing nodes by solving:

$$\sum_{i=1}^{N}\mathbf{S}'_{[k,i]}p_i = \mathbf{S}'_c\mathbf{X}_{c[j,:]} \quad (j = N+1, \ldots, N+M) \tag{44}$$

This distributes the subgraph's effect across the original graph through prompts.

G.1.2 Necessity. Prove that if any node lacks a prompt ($\exists j, p_j = 0$), there exists $\psi(\cdot)$ that cannot be simulated. First, we define a specific $\psi(\cdot)$ as: 1) Add an edge (v_j, v_k) to $\mathbf{A}$, resulting in $\hat{\mathbf{A}}$. 2) Modify $\mathbf{X}_{[j,:]} \to \mathbf{X}_{[j,:]} + \delta$, where $\delta \neq 0$. To simulate $\psi(\cdot)$ by universal graph prompts, we need:

$$f_\theta(\mathbf{A}, \mathbf{X} + p) = f_\theta(\hat{\mathbf{A}}, \mathbf{X} + \delta\mathbf{e}_j), \tag{45}$$

where $\mathbf{e}_j$ is a one-hot vector. Expanding both sides:

$$f_\theta(\mathbf{A}, \mathbf{X}+p) = \mathbf{S}(\mathbf{X}+p)\mathbf{W} = (\mathbf{S}+\Delta\mathbf{S})\left(\mathbf{X} + \delta\mathbf{e}_j\right)\mathbf{W} = f_\theta(\hat{\mathbf{A}}, \mathbf{X}+\delta\mathbf{e}_j). \tag{46}$$

Equating the two:

$$\mathbf{S}p\mathbf{W} = \Delta\mathbf{S}\mathbf{X}\mathbf{W} + \Delta\mathbf{S}\delta\mathbf{e}_j\mathbf{W} + \mathbf{S}\delta\mathbf{e}_j\mathbf{W}. \tag{47}$$

Dividing both sides by $\mathbf{W}$ (full-rank):

$$\mathbf{S}p = \Delta\mathbf{S}\mathbf{X} + (\Delta\mathbf{S} + \mathbf{S})\delta\mathbf{e}_j. \tag{48}$$

If $p_j = 0$ (j-th row of p is zero). From the equation $\mathbf{S}p = \Delta\mathbf{S}\mathbf{X} + (\Delta\mathbf{S} + \mathbf{S})\delta\mathbf{e}_j$, the j-th row gives:

$$\sum_k \mathbf{S}_{[j,k]}p_k = \sum_k \Delta\mathbf{S}_{[j,k]}\mathbf{X}_{[k,j]} + (\Delta\mathbf{S}_{[j,j]} + \mathbf{S}_{[j,j]})\delta. \tag{49}$$

Since $p_k = 0$ for all k, the left-hand side is zero. However, 1)$\Delta\mathbf{S}_{[j,j]} = 0$ (adding an edge does not change self-loops) and 2) $\mathbf{S}_{[j,j]} = 1+\epsilon \neq 0$. Thus:

$$0 = \sum_k \Delta\mathbf{S}_{[j,k]}\mathbf{X}_{[k,:]} + (1+\epsilon)\delta. \tag{50}$$

This implies $\delta = -\frac{1}{1+\epsilon}\sum_k \Delta\mathbf{S}_{[j,k]}\mathbf{X}_{[k,:]}$, which contradicts the arbitrary choice of $\delta \neq 0$.

Extension to Multi-Layer Scenario

Construct $\psi(\cdot)$ by: 1) Add an edge (v_j, v_k) to $\mathbf{A}$, resulting in $\hat{\mathbf{A}}$ and modifying all $\mathrm{S}_{(l)}$ to $\mathrm{S}'_{(l)} = \hat{\mathbf{A}} + (1+\epsilon_l)\mathbf{I}$. 2) Modify node features: $\mathbf{X}_{[j,:]} \to \mathbf{X}_{[j,:]} + \delta$, where $\delta \neq 0$. The output is:

$$\mathbf{H}'_{(L)} = \left(\prod_{l=1}^{L} \mathrm{S}'_{(l)}\right)(\mathbf{X} + \delta \mathbf{e}_j). \tag{51}$$

To simulate $\mathbf{H}'_{(L)}$, enforce:

$$\underbrace{\left(\prod_{l=1}^{L} \mathrm{S}_{(l)}\right)}_{S'}(\mathbf{X} + p). \tag{52}$$

Canceling $\mathbf{W}'$ (full-rank):

$$\mathrm{S}'(\mathbf{X} + p) = \left(\prod_{l=1}^{L} \mathrm{S}'_{(l)}\right)(\mathbf{X} + \delta \mathbf{e}_j). \tag{53}$$

Expanding the right-hand side:

$$\mathrm{S}''\mathbf{X} + \mathrm{S}''\delta \mathbf{e}_j, \quad \mathrm{S}'' = \prod_{l=1}^{L} \mathrm{S}'_{(l)}. \tag{54}$$

Rearranging terms:

$$\mathrm{S}'p = (\mathrm{S}'' - \mathrm{S}')\,\mathbf{X} + \mathrm{S}''\delta \mathbf{e}_j. \tag{55}$$

For node v_j, the j-th row equation is:

$$\sum_{n=1}^{N} \mathrm{S}'_{[j,n]} \boldsymbol{p}_n = \sum_{n=1}^{N} \left(\mathrm{S}''_{[j,n]} - \mathrm{S}'_{[j,n]}\right) \mathbf{X}_{[n,:]} + \mathrm{S}''_{[j,j]}\delta. \tag{56}$$

If node v_j lacks a prompt ($p_j = 0$), the equation reduces to:

$$\sum_{n \neq j} \mathrm{S}'_{[j,n]} p_n = \sum_{n=1}^{N} \left(\mathrm{S}''_{[j,n]} - \mathrm{S}'_{[j,n]}\right) \mathbf{X}_{[n,:]} + \mathrm{S}''_{[j,j]}\delta. \tag{57}$$

This implies:

$$\mathrm{S}''_{[j,j]}\delta = -\sum_{n=1}^{N} \left(\mathrm{S}''_{[j,n]} - \mathrm{S}'_{[j,n]}\right) \mathbf{X}_{[n,j]}, \tag{58}$$

which forces δ to depend on $\mathbf{X}$, contradicting the arbitrary choice of $\delta \neq 0$.

G.2 Connection to Recent Theoretical Work

A recent theoretical work [30], which proves that any prompt design preserves the GPF foundation as long as the model's weight matrix is row full-rank. In practice, however, pre-trained GNNs seldom satisfy the row full-rank condition. Although that study offers solid theoretical insights, it provides limited practical guidance for future research directions.

Our Theorem 1 and their results mutually reinforce each other. Specifically, when the pre-trained graph does not satisfy the row full-rank condition, selectively adding prompts to only a subset of nodes may prevent the model from achieving row full-rank. In contrast, if prompts are added to all nodes, a solution theoretically exists that ensures the graph satisfies the row full-rank condition, thereby preserving the GPF foundation. Our "Learning and Editing" paradigm offers a concrete and actionable solution aligned with this theoretical insight.